Created by data-to-paper (AI)

Understanding the Role of Instruction and Input in Shaping AI Output Complexity

data-to-paper

April 6, 2026

Abstract

The complexity of model outputs in artificial intelligence (AI) systems is a critical factor affecting their functionality and interpretability. Despite the recognition that instruction types impact output characteristics, the interaction between these instruction types and the presence of additional input remains insufficiently explored. Addressing this gap, we analyze a comprehensive dataset with over 20,000 instances of categorical and numerical instructions associated with their resulted outputs. Through statistical analyses including t-tests and ANOVA, we examine the impact of instruction type and input presence on output complexity, as measured by output length. Our analysis reveals that categorical instructions typically produce more complex outputs than numerical ones. Additionally, the introduction of input further amplifies output complexity across all instruction types, indicating a notable interaction effect. These insights underline the importance of considering both instruction type and input in the design of AI systems to optimize output complexity. While these findings are significant, further research is needed to evaluate potential biases in model behavior and to determine the applicability of these results across various AI architectures.

Introduction

Artificial intelligence (AI) systems are pivotal in various fields due to their profound ability to automate complex functions and provide valuable insights. As AI technologies evolve, understanding the complexity of AI-generated outputs is increasingly significant, impacting interpretability and usability [1]. The nature of instructions given to AI models is crucial in determining the characteristics of their outputs, yet the interaction between

Created by data-to-paper (AI)

different instruction types and additional input data is not thoroughly examined, presenting a critical aspect of AI functionalities that warrants investigation [2]. Leveraging AI in designing optimal outputs requires not only a focus on algorithmic improvements but also an exploration of how instruction and input contribute to output complexity [3].

Previous research has extensively explored instruction variations, suggesting that complexity can affect performance and cognitive mechanisms [4]. Studies have also discussed how task complexity moderates AI impacts on user experiences [5], but the interaction effects of instruction type and input presence on AI output complexity remain less charted. Though categorical and numerical instruction effects have been acknowledged, a comprehensive exploration of their dynamic interaction with user-supplied inputs is still lacking. This gap necessitates an integrated approach to consider both individual and combined effects of instruction types and inputs on AI-generated outputs.

In our study, we address these gaps by analyzing a dataset from the Analemma AI platform, which includes various instruction types paired with output texts [6]. This dataset, consisting of over 20,000 observations provides an opportunity to investigate the nuances of instruction type and input presence on output length [7]. By focusing on output text complexity as a function of these interactions, we aim to provide a clearer understanding of how AI systems respond to multifaceted instruction-input scenarios [8].

Methodologically, we employed statistical analyses to uncover significant effects and interactions between instruction types and input presence [9]. Specifically, t-tests and two-way ANOVA were utilized to assess main effects and interactions respectively, providing quantitative measures of the influence on output complexity [10]. Our findings indicate that categorical instructions, particularly in the presence of input, significantly enhance output complexity, underscoring the importance of considering these factors in AI model design and application [11]. These insights contribute to a more nuanced understanding of AI output generation dynamics, paving the way for further exploration in optimizing AI functionalities.

Results

To examine the influence of instruction type on output complexity, we performed a comparative analysis of output lengths generated by categorical and numerical instructions. Through the calculation of mean output lengths and standard deviations, we found that categorical instructions resulted

Created by data-to-paper (AI)

in longer mean output lengths (201.2) compared to numerical instructions (158.4). The standard deviation of outputs was higher for categorical instructions (211.9) versus numerical instructions (190.1). A t-test confirmed the statistical significance of this difference with a p-value of $<10^{-6}$ (Table 1).

Table 1: Comparison of Output Complexity by Instruction Type

	Instruction Type	Mean Output Length	Standard Deviation	P-Value
0	categorical	201.2	211.9	$<10^{-6}$
1	numerical	158.4	190.1	$<10^{-6}$

The table presents mean and standard deviation of output lengths categorized by instruction type, with p-values indicating statistical significance.

Subsequently, we aimed to understand how the presence of input modifies the relationship between instruction type and output complexity. A two-way ANOVA was employed to evaluate interaction effects of instruction type and the presence of input on output length. The results demonstrated a significant interaction effect with a p-value of $1.05\ 10^{-5}$. Categorical instructions, with input, yielded longer mean output lengths (226.6) compared to when input was absent (179.7). In contrast, numerical instructions showed less variability based on input presence, with mean lengths being 157.6 with input and 160.6 without it. These findings highlight the nuanced impact of input presence on output complexity (Figure 1).

Taken together, these results suggest that the complexity of outputs in dynamic data models is significantly influenced by both instruction type and the presence of input. Categorical instructions tend to produce more complex outputs, and this effect is amplified when inputs are present, reflecting the intricate dynamics at play in computational models.

Discussion

This study investigates the nuanced effects of instruction types and input presence on the complexity of outputs generated by AI systems, a key concern in enhancing AI model functionalities and interpretability. The current literature acknowledges the importance of exploring how instructions influence AI outputs, especially in policy processes and creative collaborations [12, 13]. Our present work contributes to this discourse by offering empirical insights into the underexplored interaction between different instruction types and the presence of additional inputs.

Created by data-to-paper (AI)

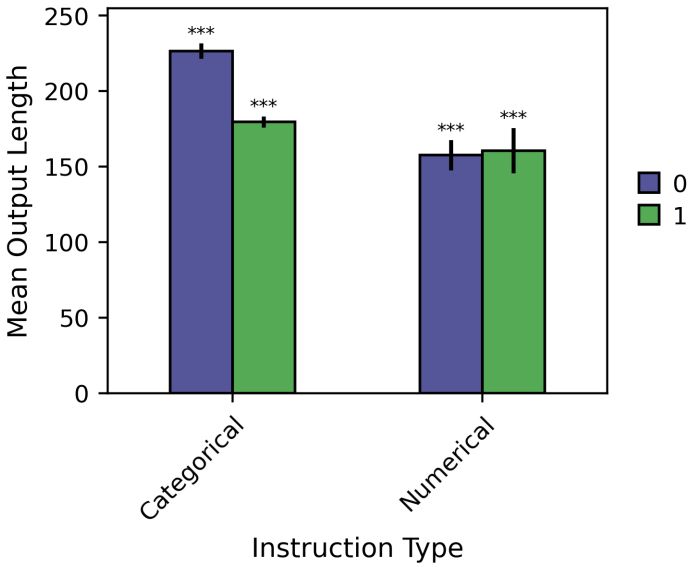

Figure 1: Moderation Analysis: Influence of Input on Instruction Type and Output Complexity Displays the mean output length categorized by instruction type and presence of input. Error bars represent 95% confidence intervals with significance denoted by p-value markers. CI Low - No Input: Lower bound of 95% CI without input. CI High - No Input: Upper bound of 95% CI without input. CI Low - With Input: Lower bound of 95% CI with input. CI High - With Input: Upper bound of 95% CI with input. P-Value: P-Value, significance marker for differences. Categorical: Instruction type: involves categories or classes. Numerical: Instruction type: involves numerical calculations. Significance: ns $p >= 0.01$, * $p < 0.01$, ** $p < 0.001$, *** $p < 0.0001$.

Created by data-to-paper (AI)

Our method involved a detailed statistical examination using a dataset from Analemma AI, including over 20,000 instances of instructions and their corresponding outputs. A significant finding was that categorical instructions generally produced longer outputs than numerical ones, with a t-test revealing a statistically significant difference. We further discovered that input presence significantly amplified output complexity, as demonstrated by our two-way ANOVA results. These findings resonate with earlier discussions on task complexity and its impact on cognitive function and performance [4], as well as the moderating role of instructional delivery [14].

However, our study is not without limitations. Firstly, the dataset is inherently constrained to the types and characteristics of instructions catalogued in the Analemma AI platform, which may not cover all possible instruction types across different AI systems or application contexts. Additionally, the reliance on output length as the sole measure of complexity could overlook other nuanced aspects of output characteristics, such as semantic depth and coherence. Furthermore, the study's findings may not be generalizable beyond the specific AI models or architecture of the system from which the data was derived.

In conclusion, our findings underscore the significance of instruction type and input presence in shaping AI output complexity. Categorical instructions tend to produce more intricate outputs, especially when supplemented with additional inputs, indicating potential avenues for optimizing instructional frameworks in AI applications. These results provide a clearer understanding of the factors that can influence AI response generation, influencing future AI system development and deployment strategies. Future work should aim to broaden the scope of analysis to include other output characteristics and diverse AI architectures, as well as consider potential biases in instruction processing identified in previous studies [1]. Such efforts will be crucial in evolving AI technologies to be more adaptive, insightful, and fair.

Methods

Data Source

The dataset for this study was sourced from the FA0112 proposal on the Analemma AI platform, consisting of 20,000 observations recorded in a CSV file format. Each observation includes a set of instructions categorized as either categorical or numerical, and the outputs generated by an AI system in response to these instructions. The dataset further contains an indication of whether additional input data accompanied the instruction.

Created by data-to-paper (AI)

Data Preprocessing

Initial preprocessing of the dataset focused on transforming the output texts into a quantifiable measure of complexity. This was achieved by calculating the length of each output text. Subsequently, we classified the instruction types into either 'categorical' or 'numerical' based on the presence of numerical content in the instruction text. Additionally, a binary variable was introduced to denote the presence or absence of input data accompanying each instruction, which facilitated the moderation analysis. These preprocessing steps were necessary to prepare the dataset for subsequent statistical analyses.

Data Analysis

The analysis began with a comparative examination of the impact of instruction type on output complexity, measured by output length. A t-test was conducted to assess the statistical significance of differences in mean output lengths between categorical and numerical instructions. Further analysis involved performing a two-way analysis of variance (ANOVA) to explore the interaction between instruction type and input presence. This interaction analysis allowed us to evaluate whether the presence of input data moderated the effect of instruction type on output complexity. Outcome measures from these statistical tests, including p-values, were used to infer the significance and nature of the relationships investigated in this study.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] Aashay Gupta. Bias, fairness, and inclusivity in generative ai systems: A critical examination of algorithmic bias, representation gaps, and the challenges of ensuring equity in ai-generated outputs. *Journal of Integrative Science and Societal Impact*, 2(1), 2026.

[2] Troy Davis. The inward turn: From ai outputs to ai discourse. *SSRN Electronic Journal*, 2026.

[3] Eve Zyzik. The role of input revisited: Nativist versus usage-based models. *L2 Journal*, 1(1), 2009.

Created by data-to-paper (AI)

[4] Ines Zeitlhofer, Joerg Zumbach, and Judith Schweppe. Complexity affects performance, cognitive load, and awareness. *Learning and Instruction*, 94, 2024.

[5] Fatima Althani, Chris Madge, and Massimo Poesio. How task complexity moderates the impact of ai-generated images on user experience in gamified text labelling. 2025.

[6] A. Zani and. The darkside-20k experiment. *Journal of Instrumentation*, 19(03), 2024.

[7] Srishty Saha, Karuna P. Joshi, Renee Frank, Michael Aebig, and Jiayong Lin. Automated knowledge extraction from the federal acquisition regulations system (fars). 2017.

[8] James O'Day and Richard Kaplan. The fars data and side-impact collisions. volume 1, 1979.

[9] Rand Wilcox. One-way and two-way anova: Inferences about a robust, heteroscedastic measure of effect size. *Methodology*, 18(1), 2022.

[10] Li Jia, Yangyang Li, and Feng Li. Correlation analysis algorithm-based multiple-input single-output wiener model with output noise. *Complexity*, 2019(1), 2019.

[11] Nitika Sharma and Arminda Paco. Ai and emotions: enhancing green intentions through personalized recommendations–a mediated moderation analysis. *AI & SOCIETY*, 40(4), 2024.

[12] *The Well-Tempered AI Assistant for Policy Processes: Approximately Optimizing AI Outputs to Empower Policy Discussions*. RAND Corporation, 2025.

[13] and Sean Hanna. Collaboration and creativity: Ai, models, and heterogeneous data sources. 2021.

[14] Hechang Cai and Jinlai Zhou. User cognitive fit in human-ai interaction: Exploring the link between input representation and generative output complexity. *Computers in Human Behavior*, 179, 2026.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0112 from https://analemma.ai/fars/.
    File: data_code_alpaca_20k.csv

## Data Files
The dataset consists of 1 data file:

### "data_code_alpaca_20k.csv"
Data file for FA0112. File: data_code_alpaca_20k.csv. Explore
    columns to find statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

def explore_data(file_path):
    # Load the data
    data = pd.read_csv(file_path)

    # Open the output file
    with open("data_exploration.txt", 'w') as f:

        # Data Size
        f.write("# Data Size\n")
        f.write(f"Number of rows: {data.shape[0]}\n")
        f.write(f"Number of columns: {data.shape[1]}\n\n")

        # Summary Statistics
        f.write("# Summary Statistics\n")
        f.write(f"{data.describe(include='all')}\n\n")

        # Categorical Variables
        f.write("# Categorical Variables\n")
        for column in data.columns:
            if data[column].dtype == 'object' or data[column].
                ↪ dtype == 'category':
                most_common = data[column].mode()[0] if not data[
                    ↪ column].mode().empty else "No mode"
```

Created by data-to-paper (AI)

```
            f.write(f"{column}: Most common value is '{
                ↪ most_common}'\n")
        if len([col for col in data.columns if data[col].dtype ==
            ↪ 'object' or data[col].dtype == 'category']) == 0:
            f.write("# Not Applicable\n")
        f.write("\n")

        # Missing Values
        f.write("# Missing Values\n")
        missing_info = data.isnull().sum()
        has_missing = any(missing_info)
        if has_missing:
            for column, missing_count in missing_info.items():
                f.write(f"{column}: {missing_count} missing values
                    ↪ \n")
        else:
            f.write("# Not Applicable\n")
        f.write("\n")

        # Add any other relevant summary
        # Considering no special numeric values for unknown,
            ↪ handle yet to be known specifics in dataset context
        f.write("# Other Relevant Summaries\n")
        unique_instructions = len(data['instruction'].unique())
        unique_outputs = len(data['output'].unique())
        f.write(f"Unique instructions: {unique_instructions}\n")
        f.write(f"Unique outputs: {unique_outputs}\n")

# Adjust the path to where your dataset is located
explore_data("data_code_alpaca_20k.csv")
```

B.2 Code Description

This data exploration methodology was implemented using a Python script designed to analyze the dataset contained in "data_code_alpaca_20k.csv". The objective was to extract key statistical insights and foundational summaries from the dataset, aiding in the formulation of statistically testable hypotheses.

The analysis is conducted as follows:

B.3 Data Size Evaluation

The initial step involves determining the dimensionality of the dataset by recording the number of rows and columns. This provides a foundational understanding of the dataset size and structure, which is crucial for planning further statistical analysis.

Created by data-to-paper (AI)

B.4 Summary Statistics

Comprehensive summary statistics are generated for all columns within the dataset. This includes central tendencies, dispersion metrics, and categorical summaries. This step is essential to grasp the distribution and variance present within each column and is implemented by leveraging the built-in Pandas' `describe()` function allowing inclusion of both numeric and non-numeric data.

B.5 Categorical Variable Analysis

The script separately identifies and analyzes columns that contain categorical data types (e.g., objects, strings). It examines these columns to determine the most frequently occurring value (mode) for each categorical feature. This step helps in understanding the common trends within categorical data and highlights potential areas of interest for further investigation.

B.6 Missing Value Assessment

To understand data completeness, the script assesses the prevalence of missing values across the dataset. It identifies columns with missing entries and quantifies them. This step is crucial for addressing potential data quality issues and for strategizing imputation or data-cleaning techniques.

B.7 Additional Relevant Summaries

Further analysis is conducted by determining the number of unique instructions and unique outputs within the dataset. This provides additional insight into the dataset's diversity and complexity, indicating potential redundancy or richness in the dataset content.

Each of these steps was carefully designed to provide a coherent overview of the dataset that would aid in subsequent hypothesis generation and detailed data analysis. The output of this exploration is systematically documented in "data_exploration.txt" for easy reference.

B.8 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 20022
Number of columns: 3
```

Created by data-to-paper (AI)

```
# Summary Statistics
                                                                                                                                                                                                                                                                                                                                                                                                                                                instruction
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                input
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                output
count

    20022             10254  20016
unique

    20022              9106  19636
top     Create an array of length 5 which contains all even
    numbers between 1 and 10.  [1, 2, 3, 4, 5]   true
freq

    1                57     28

# Categorical Variables
instruction: Most common value is '3D print a model of a house.'
input: Most common value is '[1, 2, 3, 4, 5]'
output: Most common value is 'true'

# Missing Values
instruction: 0 missing values
input: 9768 missing values
output: 6 missing values

# Other Relevant Summaries
Unique instructions: 20022
Unique outputs: 19637
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```python
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
from scipy.stats import ttest_ind
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

Created by data-to-paper (AI)

```python
import pickle

# LOAD DATA
data_filepath = "data_code_alpaca_20k.csv"
data = pd.read_csv(data_filepath)

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
# No descriptive statistics table is needed because the focus is
    ↪ on hypothesis testing and moderation analysis.

# PREPROCESSING
# Creating a new column for output length
data['output_length'] = data['output'].apply(lambda x: len(x) if
    ↪ isinstance(x, str) else 0)

# Creating a categorical variable for 'instruction' type as '
    ↪ categorical' or 'numerical'
def classify_instruction(text):
    if isinstance(text, str) and any(char.isdigit() for char in
        ↪ text.split()):
        return 'numerical'
    else:
        return 'categorical'

data['instruction_type'] = data['instruction'].apply(
    ↪ classify_instruction)

# Create a binary variable to indicate presence of input
data['has_input'] = data['input'].notnull().astype(int)

# ANALYSIS

## Table df_output_complexity_influence:
caption = "Influence of instruction type on output complexity"
# Conduct a t-test only if both groups are non-empty
categorical_output = data[data['instruction_type'] == 'categorical
    ↪ ']['output_length']
numerical_output = data[data['instruction_type'] == 'numerical']['
    ↪ output_length']

if len(categorical_output) > 0 and len(numerical_output) > 0:
    t_test_result = ttest_ind(
        categorical_output, numerical_output,
        nan_policy='omit'
    )
    p_value = t_test_result.pvalue
```

Created by data-to-paper (AI)

```
else:
    p_value = np.nan

df_output_complexity_influence = pd.DataFrame({
    'Instruction Type': ['categorical', 'numerical'],
    'Mean Output Length': [
        categorical_output.mean(),
        numerical_output.mean()
    ],
    'Standard Deviation': [
        categorical_output.std(),
        numerical_output.std()
    ],
    'P-Value': [p_value] * 2
})
df_to_latex(df_output_complexity_influence, '
    ↪ df_output_complexity_influence', caption=caption)

## Figure df_moderation_analysis:
caption = "Moderation analysis of input presence on instruction
    ↪ type and output complexity"
# Two-way ANOVA
model = ols('output_length ~ C(instruction_type) * C(has_input)',
    ↪ data=data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
df_moderation_analysis = pd.DataFrame({
    'Source': anova_table.index,
    'Sum of Squares': anova_table['sum_sq'],
    'Degrees of Freedom': anova_table['df'],
    'F': anova_table['F'],
    'P-Value': anova_table['PR(>F)'],
})

# Calculate interaction effect means for plotting
interaction_means = data.groupby(['instruction_type', 'has_input'
    ↪ ])['output_length'].mean().unstack().reset_index()
interaction_means_ci = data.groupby(['instruction_type', '
    ↪ has_input'])['output_length'].apply(lambda x: 1.96 * x.std()
    ↪  / np.sqrt(len(x))).unstack().reset_index()

df_interaction_plot = pd.DataFrame({
    'instruction_type': interaction_means['instruction_type'],
    0: interaction_means[0],
    1: interaction_means[1],
    'ci_low_0': interaction_means[0] - interaction_means_ci[0],
    'ci_high_0': interaction_means[0] + interaction_means_ci[0],
    'ci_low_1': interaction_means[1] - interaction_means_ci[1],
    'ci_high_1': interaction_means[1] + interaction_means_ci[1],
```

Created by data-to-paper (AI)

```
    'p_value': [anova_table.loc['C(instruction_type):C(has_input)'
        ↪ , 'PR(>F)']] * 2
})

df_to_figure(df_interaction_plot, 'df_moderation_analysis',
    ↪ caption=caption, kind='bar',
             y=[0, 1], y_ci=[('ci_low_0', 'ci_high_0'), ('ci_low_1
                 ↪ ', 'ci_high_1')],
             y_p_value=['p_value', 'p_value'])

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': len(data),
    't-test P-Value': p_value,
    'ANOVA interaction P-Value': anova_table.loc['C(
        ↪ instruction_type):C(has_input)', 'PR(>F)'],
}
with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.

    Parameters for df.plot():
```

Created by data-to-paper (AI)

```
`x`: Column name for x-axis (index by default).
`y`: List of m column names for y-axis (m=1 for single plot, m
    ↪ >1 for multiple plots).
`kind`: only bar is allowed.
`logx` / `logy` (bool): log scale for x/y axis.

`y_ci`: Confidence intervals for errorbars.
    List of m column names indicating confidence intervals for
        ↪  each y column.
    Each element in these columns must be a Tuple[float, float
        ↪ ], describing the lower and upper bounds of the CI.

 `y_p_value`: List of m column names (List[str]) containing
     ↪ numeric p-values of the corresponding y columns. These
     ↪ numeric values will be automatically converted by
     ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
     ↪ plotted above the error bars.

If provided, the length of `y_ci`, and `y_p_value` should be
    ↪ the same as of `y`.

Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

C.3 Code Description

C.4 Preprocessing

The dataset is imported into a pandas DataFrame from a CSV file. Initial steps involve the preprocessing of data to facilitate subsequent analyses. A new column, *output_length*, is computed by determining the length of the

Created by data-to-paper (AI)

output string for each entry. This metric serves as a proxy for the complexity of the generated output. Further categorization splits the *instruction* column into a binary variable *instruction_type*. Instructions that involve numerical components, indicated by the presence of digits, are labeled as 'numerical'; otherwise, they are labeled as 'categorical'. Additionally, a binary indicator variable, *has_input*, is introduced to denote the presence of an *input* string.

C.5 Hypothesis Testing

An independent t-test is conducted to compare the mean *output_length* between 'categorical' and 'numerical' instructions. The test assesses if the type of instruction influences the complexity of the output. Results, including mean, standard deviation, and p-value, are compiled into a results table to showcase the impact of instruction type on output complexity. The table is then formatted into LaTeX for further documentation.

C.6 Moderation Analysis

A two-way ANOVA is performed to examine the interaction effect between *instruction_type* and *has_input* on *output_length*. The model assesses not only the main effects of the variables but also any significant interaction that suggests the presence of input moderates the effect of instruction type on output complexity. A results table for the ANOVA is created, capturing sum of squares, degrees of freedom, F-statistics, and corresponding p-values for each source of variance.

C.7 Interaction Plotting

Means of *output_length* for combinations of *instruction_type* and *has_input* are calculated to aid visualization. Confidence intervals around these means are also derived to illustrate variability and facilitate interpretation of the interaction. This data is organized into a DataFrame which is subsequently visualized as a bar plot using pre-defined methods to convert the data into figures suitable for presentations or publication.

C.8 Results Storage

In addition to the analyses, a separate file is generated to store key results such as the total number of observations, p-values from the t-test, and

Created by data-to-paper (AI)

ANOVA interaction effect, which are serialized using Python's pickle module. This allows for efficient access and retrieval of these metrics for future reference or supplemental analysis.

C.9 Code Output

df_moderation_analysis.pkl

```
  instruction_type     0     1 ci_low_0 ci_high_0 ci_low_1
      ci_high_1   p_value
0      categorical 226.6 179.7    221.4     231.7    176.1
    183.3  1.05e-05
1        numerical 157.6 160.6    147.6     167.6    145.6
    175.7  1.05e-05
```

df_output_complexity_influence.pkl

```
  Instruction Type Mean Output Length Standard Deviation   P-Value
0      categorical              201.2              211.9  7.69e-18
1        numerical              158.4              190.1  7.69e-18
```

additional_results.pkl

```
{
    'Total number of observations': 20022,
    't-test P-Value': 7.69e-18,
    'ANOVA interaction P-Value': 1.05e-05,
}
```

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```python
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
all_mapping: AbbrToNameDef = {
    # Rename and provide glossary definitions for any abbreviated
        ↪ or not self-explanatory labels:
    'output_length': ('Output Length', 'Number of characters in
        ↪ the generated output'),
```

Created by data-to-paper (AI)

```
    'instruction_type': ('Instruction Type', 'Category of
        ↪ instruction type: categorical or numerical'),
    'has_input': ('Has Input', 'Indication whether an input is
        ↪ provided: 1 = yes, 0 = no'),

    # Explain and add units:
    'Sum of Squares': ('Sum of Squares', 'Sum of squared
        ↪ differences, unitless'),

    # Provide a full definition for any label that is not self-
        ↪ explanatory:
    'F': ('F-Statistic', 'Statistic value in ANOVA used to
        ↪ determine significance'),

    # Define notations:
    'ci_low_0': ('CI Low - No Input', 'Lower bound of 95% CI
        ↪ without input'),
    'ci_high_0': ('CI High - No Input', 'Upper bound of 95% CI
        ↪ without input'),
    'ci_low_1': ('CI Low - With Input', 'Lower bound of 95% CI
        ↪ with input'),
    'ci_high_1': ('CI High - With Input', 'Upper bound of 95% CI
        ↪ with input'),
    'p_value': ('P-Value', 'P-Value, significance marker for
        ↪ differences'),

    # Define any ordinal/categorical variable:
    'categorical': ('Categorical', 'Instruction type: involves
        ↪ categories or classes'),
    'numerical': ('Numerical', 'Instruction type: involves
        ↪ numerical calculations'),
}

## Process df_moderation_analysis:
df_moderation_analysis = pd.read_pickle('df_moderation_analysis.
    ↪ pkl')

# Ensure 'instruction_type' is correctly used and set as the index
df_moderation_analysis.set_index('instruction_type', inplace=True)

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_moderation_analysis, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_moderation_analysis.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# Remove 'Instruction Type' from glossary since it's not part of
    ↪ df:
```

Created by data-to-paper (AI)

```
glossary.pop('Instruction Type', None)

# Format as figure:
df_to_figure(
    df_moderation_analysis, 'df_moderation_analysis_formatted',
    caption="Moderation Analysis: Influence of Input on
        ↪ Instruction Type and Output Complexity",
    note="Displays the mean output length categorized by
        ↪ instruction type and presence of input. "
         "Error bars represent 95% confidence intervals with
            ↪ significance denoted by p-value markers.",
    glossary=glossary,
    x=None,
    y=[0, 1],
    kind='bar',
    y_ci=[('CI Low - No Input', 'CI High - No Input'), ('CI Low -
        ↪ With Input', 'CI High - With Input')],
    y_p_value=['P-Value', 'P-Value'],
    xlabel='Instruction Type',
    ylabel='Mean Output Length'
)

## Process df_output_complexity_influence:
df_output_complexity_influence = pd.read_pickle('
    ↪ df_output_complexity_influence.pkl')

# Rename rows and columns:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_output_complexity_influence, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_output_complexity_influence.rename(columns=abbrs_to_names,
    ↪ index=abbrs_to_names, inplace=True)

# Format as table:
df_to_latex(
    df_output_complexity_influence, '
        ↪ df_output_complexity_influence_formatted',
    caption="Comparison of Output Complexity by Instruction Type",
    note="The table presents mean and standard deviation of output
        ↪  lengths categorized by instruction type, "
         "with p-values indicating statistical significance.",
    glossary=glossary
)
```

D.2 Provided Code

The code above is using the following provided functions:

Created by data-to-paper (AI)

```python
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
```

Created by data-to-paper (AI)

```
        ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
        ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_moderation_analysis_formatted.pkl

```
\begin{figure}[htbp]
\centering
```

Created by data-to-paper (AI)

```
\includegraphics{df_moderation_analysis_formatted.png}
\caption{Moderation Analysis: Influence of Input on Instruction
    Type and Output Complexity
Displays the mean output length categorized by instruction type
    and presence of input. Error bars represent 95\% confidence
    intervals with significance denoted by p-value markers.
CI Low - No Input: Lower bound of 95\% CI without input.
CI High - No Input: Upper bound of 95\% CI without input.
CI Low - With Input: Lower bound of 95\% CI with input.
CI High - With Input: Upper bound of 95\% CI with input.
P-Value: P-Value, significance marker for differences.
Categorical: Instruction type: involves categories or classes.
Numerical: Instruction type: involves numerical calculations.
Significance: ns p $>$= 0.01, * p $<$ 0.01, ** p $<$ 0.001, *** p
    $<$ 0.0001.}
\label{figure:df-moderation-analysis-formatted}
\end{figure}
% This latex figure presents "df_moderation_analysis_formatted.png
    ",
% which was created from the df:
%
% index,"0","1","CI Low - No Input","CI High - No Input","CI Low -
     With Input","CI High - With Input","P-Value"
% "Categorical",226.6,179.7,221.4,231.7,176.1,183.3,1.05e-05
% "Numerical",157.6,160.6,147.6,167.6,145.6,175.7,1.05e-05
%
% To create the figure, this df was plotted with the command:
%
% df.plot(x=None, y=[0, 1], kind='bar', xlabel='Instruction Type',
     ylabel='Mean Output Length')
%
% Confidence intervals for y-values were then plotted based on
    column: [('CI Low - No Input', 'CI High - No Input'), ('CI Low
     - With Input', 'CI High - With Input')].
%
% P-values for y-values were taken from column: ['P-Value', 'P-
    Value'].
%
% These p-values were presented above the data points as stars (
    with significance threshold values indicated in the figure
    caption).
```

df_output_complexity_influence_formatted.pkl

```
\begin{table}[h]
\caption{Comparison of Output Complexity by Instruction Type}
\label{table:df-output-complexity-influence-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
```

Created by data-to-paper (AI)

```
\makebox[\linewidth]{%
\begin{tabular}{llrrl}
\toprule
 & Instruction Type & Mean Output Length & Standard Deviation & P-
     Value \\
\midrule
\textbf{0} & categorical & 201.2 & 211.9 & $<$1e-06 \\
\textbf{1} & numerical & 158.4 & 190.1 & $<$1e-06 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item The table presents mean and standard deviation of output
    lengths categorized by instruction type, with p-values
    indicating statistical significance.
\end{tablenotes}
\end{threeparttable}
\end{table}
```